

ESCUELA TECNICA SUPERIOR DE INGENIERIA INFORMATICA

DOBLE GRADO EN INGENIERIA INFORMATICA E INGENIERIA DE  
COMPUTADORES

CURSO ACADEMICO 2025-2026

TRABAJO FIN DE GRADO INGENIERÍA DE COMPUTADORES

TELEGRAM COMO FUENTE DE INTELIGENCIA EN  
CIBERSEGURIDAD: PIPELINE DE DETECCION DE PHISHING Y  
MENSAJES SOSPECHOSOS PARA ENTORNOS EMPRESARIALES

Autor: Alvaro Osuna Flores

Tutora: Liliana Patricia Santacruz Valencia

Resumen 1

1. Introduccion 2

1.1 Motivacion 2

1.2 Objetivos 3

1.3 Estructura del documento 4

2. Contexto 4

2.1 Telegram como superficie de interes en ciberseguridad 4

2.2 Necesidad de un enfoque de sistemas 5

2.3 Soluciones relacionadas y hueco del proyecto 5

2.4 Criterio de seguridad minima 6

3. Metodologia, requisitos y arquitectura 6

3.1 Metodologia de trabajo 7

|     |                                          |    |
|-----|------------------------------------------|----|
| 3.2 | Requisitos funcionales                   | 7  |
| 3.3 | Requisitos no funcionales                | 8  |
| 3.4 | Arquitectura general del pipeline        | 8  |
| 3.5 | Flujo operativo y trazabilidad           | 9  |
| 3.6 | Herramientas y tecnologías               | 10 |
| 3.7 | Infraestructura y despliegue             | 11 |
| 4.  | Desarrollo e integracion del sistema     | 12 |
| 4.1 | Ingesta y gestion de sesion con Telegram | 12 |
| 4.2 | Preprocesado e inferencia                | 13 |
| 4.3 | Persistencia y minimizacion de datos     | 13 |
| 4.4 | Versionado de artefactos por ejecucion   | 15 |
| 4.5 | API y capa de consulta                   | 15 |
| 4.6 | Dashboard React y Grafana                | 16 |
| 4.7 | Validacion integrada                     | 17 |
| 5.  | Evaluacion y analisis de resultados      | 18 |
| 5.1 | Metricas principales                     | 18 |
| 5.2 | Interpretacion del umbral operativo      | 19 |
| 5.3 | Matriz de confusion y distribucion       | 19 |
| 5.4 | Evidencia de validacion integrada        | 20 |
| 5.5 | Limitaciones detectadas                  | 20 |
| 6.  | Conclusiones y lineas futuras            | 21 |
|     | Bibliografia                             | 22 |

## Resumen

Este Trabajo Fin de Grado presenta el diseño e integración de un sistema desacoplado para detectar mensajes de phishing y comunicaciones sospechosas procedentes de Telegram. El proyecto se aborda desde Ingeniería de Computadores como un pipeline operativo completo, en el que la captura, la cola de mensajería, la inferencia, la persistencia, la API y la observabilidad forman una misma cadena técnica.

La solución combina Telethon para la escucha de mensajes, RabbitMQ para desacoplar la ingesta, un worker de inferencia basado en DistilBERT, MongoDB para la persistencia trazable, FastAPI como contrato de consulta, un dashboard React para revisión funcional y Prometheus con Grafana para observabilidad en tiempo real.

Los artefactos de evaluación se versionan por `run_id` y los benchmarks por `benchmark_id`, lo que permite relacionar métricas, predicciones, matriz de confusión, evidencias de validación y pruebas de carga con una ejecución concreta. En la evaluación offline de referencia se obtuvo `accuracy = 0.63`, `precision_pos = 0.5823`, `recall_pos = 0.92`, `f1_pos = 0.7132` y, en el benchmark controlado más exigente, el sistema mantuvo 19.08 mensajes/s reales con una latencia media E2E de 52.4 ms y p95 de 66.06 ms.

Lo más valioso del proyecto, visto desde Ingeniería de Computadores, es que cada fase del ciclo de vida del mensaje queda conectada con la siguiente y, además, queda medida. La entrada en Telegram, la publicación en cola, el consumo por el worker, la escritura en MongoDB, la exposición por API y la monitorización en Prometheus/Grafana generan evidencia técnica útil para auditar latencias, uso de recursos y capacidad del pipeline.

Aunque el entrenamiento del modelo no constituye el eje principal de esta memoria, tampoco se ha tratado la IA como una caja negra tomada al azar. El sistema se apoya en un modelo derivado de `distilbert-base-uncased` con metadatos de entrenamiento documentados, checkpoints conservados y una ruta de carga

preparada para funcionar tanto con artefactos locales como con el repositorio versionado del proyecto.

La principal aportacion del trabajo no es solo el clasificador, sino la integracion de productor, cola, worker, persistencia, API y observabilidad en un flujo trazable y defendible, orientado a un escenario realista de operacion y pensado para absorber picos de carga y facilitar el mantenimiento.

Palabras clave: Telegram, phishing, RabbitMQ, Prometheus, FastAPI, MongoDB, observabilidad, pipeline, DistilBERT.

## 1. Introduccion

### 1.1 Motivacion

Telegram se ha convertido en un entorno de interes para la ciberseguridad por dos motivos complementarios. Por un lado, es una plataforma ampliamente utilizada para comunicacion legitima, coordinacion y difusion rapida de mensajes. Por otro, sus canales, grupos y dinamica de propagacion facilitan tambien la distribucion de enlaces fraudulentos, senuelos de phishing, credenciales comprometidas o conversaciones relevantes para la inteligencia de amenazas (Europol, 2022; Guo et al., 2024).

Desde una perspectiva operacional, el problema no consiste solo en clasificar texto. El reto real es construir un sistema capaz de recibir mensajes en tiempo casi real, tratarlos de manera consistente, decidir con que evidencia se almacenan, exponer resultados a otros componentes y mantener observabilidad suficiente para su revision posterior. Esa naturaleza integrada hace que el proyecto encaje especialmente bien en Ingenieria de Computadores.

Ademas, un analisis manual continuo resulta costoso, poco escalable y propenso a errores. En consecuencia, es razonable automatizar la primera clasificacion, siempre que el sistema deje trazabilidad tecnica suficiente para justificar la salida producida y para explicar como se ha llegado a ella.

Esta forma de plantearlo ayuda tambien a diferenciar con claridad este trabajo del futuro TFG de fake news. Mientras aquel puede

centrarse con mayor naturalidad en datos, preprocesado, entrenamiento, evaluacion y logica de aplicacion, aqui el peso recae en la arquitectura del pipeline, en la integracion entre modulos y en las decisiones de ejecucion, almacenamiento y observabilidad necesarias para operar el sistema.

## 1.2 Objetivos

Los objetivos planteados para este trabajo son los siguientes.

### Objetivo general

Desarrollar un pipeline modular para capturar mensajes de Telegram, publicarlos en una cola RabbitMQ, consumirlos desde un worker de inferencia y persistir evidencia tecnica suficiente para consultar, monitorizar y evaluar el sistema de forma reproducible.

En esta memoria, ese objetivo se entiende ademas como el diseno de un sistema desplegable, auditable y medible. Por eso, ademas del resultado de clasificacion, resulta importante justificar como se desacoplan los componentes, como se observan las latencias por etapa y como se valida el comportamiento con benchmark y pruebas.

### Objetivos especificos

Implementar la ingesta automatica de mensajes desde Telegram y desacoplarla mediante una cola RabbitMQ.

Integrar un worker de preprocesado e inferencia con un modelo binario entrenado previamente.

Persistir evidencia tecnica suficiente en MongoDB, incluyendo latencias por etapa y snapshots de recursos, sin almacenar por defecto mas datos sensibles de los estrictamente necesarios.

Versionar los artefactos de evaluacion y benchmark por `run_id` y `benchmark_id` para mantener coherencia entre ejecuciones, metricas y endpoints.

Exponer resultados mediante una API reutilizable, metricas Prometheus y visualizaciones en React y Grafana.

Validar el sistema con pruebas de API, benchmark controlado y casos simulados de integracion.

Definir una topologia de ejecucion reproducible mediante contenedores y perfiles de despliegue, separando productor, cola, worker, API y observabilidad.

Documentar la procedencia del modelo y las principales decisiones de arquitectura para justificar la integracion de IA, la eleccion de RabbitMQ, MongoDB y Prometheus y el enfoque de sistemas adoptado.

### 1.3 Estructura del documento

La memoria se organiza en seis bloques. En primer lugar se presenta el contexto del problema y la justificacion tecnica del enfoque adoptado. A continuacion se describen la metodologia, los requisitos y la arquitectura desacoplada. Despues se detalla la integracion del productor, la cola, el worker, la API y la observabilidad. Finalmente se analizan los resultados offline y del benchmark, y se cierran las conclusiones y lineas futuras.

## 2. Contexto

### 2.1 Telegram como superficie de interes en ciberseguridad

La literatura reciente y los informes del sector coinciden en que Telegram se ha consolidado como una fuente dual: sirve tanto para comunicaciones legitimas como para actividades ilicitas y de coordinacion criminal (Europol, 2022; Guo et al., 2024). Esa dualidad lo convierte en un espacio util para la inteligencia temprana en ciberseguridad.

Para una empresa, esto tiene una consecuencia clara: conviene disponer de mecanismos que permitan observar, filtrar y priorizar mensajes potencialmente peligrosos sin depender exclusivamente de la revision manual. De forma particular, el phishing sigue siendo una de las amenazas mas frecuentes y mas efectivas, por lo que detectar indicadores tempranos en mensajes o enlaces compartidos puede reducir el tiempo de reaccion.

## 2.2 Necesidad de un enfoque de sistemas

En este trabajo no basta con responder si un mensaje parece benigno o sospechoso. La pregunta relevante es mas amplia:

como llega el mensaje al sistema;

como se procesa y clasifica;

que datos se conservan;

como se audita la ejecucion;

como se consumen los resultados desde otros componentes.

Estas preguntas obligan a pensar el problema como un pipeline de sistemas. Por eso, el proyecto no se ha orientado solo a la mejora de un clasificador, sino al ensamblaje de varios modulos coordinados que permitan desplegar la solucion, validarla y operarla.

Mirado asi, el mensaje deja de ser un simple texto y pasa a ser una unidad de trabajo que atraviesa varias capas tecnicas. Primero hay una capa de adquisicion, responsable de mantener la sesion con Telegram y recibir eventos NewMessage. Despues entra en juego una capa de transformacion, donde el contenido se normaliza, se detecta el idioma y se prepara la entrada del modelo. A continuacion actua la capa de decision, que calcula score\_1, aplica el umbral operativo y adjunta metadatos del modelo y del dispositivo. Finalmente aparecen las capas de persistencia y explotacion, que almacenan el evento de forma pseudonimizada, lo publican mediante endpoints y lo convierten en una fuente de consulta operativa para React y Grafana.

Precisamente por eso este TFG encaja mejor en Ingenieria de Computadores. El interes academico no se agota en la calidad del clasificador, sino en la capacidad de integrar comunicaciones, almacenamiento, servicios, observabilidad y controles minimos de seguridad en un sistema coherente.

## 2.3 Soluciones relacionadas y hueco del proyecto

Existen soluciones comerciales y academicas para monitorizar

fuentes abiertas, orquestar indicadores o aplicar modelos de lenguaje a ciberseguridad. Sin embargo, muchas de esas soluciones son generalistas, costosas o estan poco centradas en una integracion ligera y modular sobre Telegram. El hueco que cubre este TFG consiste en demostrar una arquitectura defendible, con tecnologias accesibles y con especial atencion a:

desacoplamiento ligero de la ingesta mediante RabbitMQ;

almacenamiento operativo flexible en MongoDB;

contrato de consulta y explotacion mediante API;

observabilidad con Prometheus y Grafana;

trazabilidad por ejecucion y benchmark.

## 2.4 Criterio de seguridad minima

La revision de tutoria obligo a reforzar el sistema en un aspecto importante: la explotacion minima segura. A partir de esa revision se fijaron cinco criterios:

no exponer MongoDB ni la cola RabbitMQ fuera del entorno local salvo puertos controlados de demo;

exigir autenticacion simple en la API;

desactivar acceso anonimo en Grafana;

restringir CORS a origenes concretos;

minimizar los datos personales o textuales persistidos en MongoDB.

Este conjunto de decisiones no convierte el sistema en una plataforma final de produccion, pero si lo hace mas coherente y defendible desde el punto de vista tecnico. En particular, la consola de RabbitMQ queda como herramienta de demostracion y no como superficie abierta por defecto.

## 3. Metodologia, requisitos y arquitectura



### 3.1 Metodologia de trabajo

El proyecto se ha desarrollado con una metodologia incremental. Primero se construyo un nucleo funcional de captura, inferencia y almacenamiento. Posteriormente se reorganizo en torno a artefactos versionados por run\_id. Finalmente, a partir de la tutoria, se introdujeron RabbitMQ para desacoplar ingesta e inferencia, Prometheus para observabilidad real y un benchmark controlado para estudiar latencia, throughput y consumo de recursos.

### 3.2 Requisitos funcionales

Codigo

Requisito

RF-1

Capturar mensajes de Telegram y publicarlos en RabbitMQ mediante un productor desacoplado.

RF-2

Consumir la cola, preprocesar el texto y ejecutar inferencia binaria con un worker especializado.

RF-3

Guardar una traza tecnica minima y pseudonimizada en MongoDB, incluyendo latencias por etapa y recursos.

RF-4

Generar artefactos reproducibles de evaluacion y benchmark por run\_id y benchmark\_id.

RF-5

Publicar resultados mediante API y metricas Prometheus, y visualizarlos en React y Grafana.

RF-6

Validar el comportamiento con pruebas automatizadas, benchmark y casos simulados.

### 3.3 Requisitos no funcionales

Codigo

Requisito

RNF-1

La solucion debe ser modular, desplegable localmente y compatible con ejecucion por perfiles.

RNF-2

La ingesta y la inferencia deben poder desacoplarse para absorber picos temporales de carga.

RNF-3

Los resultados deben ser trazables por ejecucion, benchmark y artefacto asociado.

RNF-4

La observabilidad debe incluir metricas de latencia, throughput y recursos del proceso.

RNF-5

La persistencia por defecto debe reducir la exposicion de datos sensibles y mantener controles minimos de acceso.

### 3.4 Arquitectura general del pipeline

La arquitectura se organiza en cinco bloques:

captura de mensajes y publicacion en cola desde Telegram;

consumo, preprocesado e inferencia en el worker;

persistencia operativa y versionado de artefactos;

publicacion y consulta mediante API y dashboard React;

observabilidad y benchmark con Prometheus, Grafana y scripts controlados.

Figura 1. Arquitectura general del sistema con productor, RabbitMQ, worker, MongoDB, API, React y observabilidad.

En terminos practicos, la capa de entrada operativa ya no se limita a un script monolitico. main.py se mantiene como punto de entrada compatible con modos legacy, producer y worker, mientras que producer.py y worker.py separan explicitamente la captura desde Telegram de la inferencia y la persistencia.

Si se baja de la arquitectura logica a la infraestructura real del proyecto, esos bloques se corresponden con servicios diferenciados. El stack base levanta mongo, rabbitmq, worker, api y frontend; el perfil advanced anade el productor real de Telegram y el perfil observability incorpora Prometheus y Grafana. Esta topologia permite desacoplar picos de entrada, medir el comportamiento por etapa y escalar consumidores sin reescribir el resto del pipeline.

### 3.5 Flujo operativo y trazabilidad

El flujo real que sigue un mensaje es el siguiente:

Telegram entrega un mensaje al productor autenticado.

El productor publica un evento ligero en RabbitMQ con la informacion minima necesaria para el procesamiento.

El worker consume la cola, normaliza el texto, ejecuta la inferencia y decide la clase final segun un umbral configurable.

Se genera una traza con identificadores pseudonimizados, hash del mensaje, latencias por etapa, snapshots de recursos y metadatos del modelo.

La informacion queda disponible para persistencia, consulta por API, scraping de Prometheus y visualizacion operativa.

En la practica, este recorrido se convierte en una cadena de transformaciones muy concreta. `producer.py` valida las credenciales de Telegram y publica envelopes con timestamps de origen y cola; `worker.py` consume esos mensajes, llama a `pipeline.py` para construir el documento tecnico completo y, finalmente, persiste la evidencia en MongoDB si la escritura esta habilitada.

Este comportamiento es importante porque muestra que la trazabilidad no se anade a posteriori. La evidencia tecnica se genera en el mismo recorrido del mensaje y acompaña a la decision del modelo con tiempos de preprocesado, inferencia, espera en cola, escritura en base de datos y uso de CPU y memoria del proceso.

Figura 2. Flujo de productor, cola, worker y trazabilidad aplicado a cada mensaje procesado.

Esta secuencia es precisamente la que refuerza el enfoque de sistemas del TFG. La salida del modelo no se trata como un dato aislado, sino como un evento que atraviesa capas tecnicas distintas, absorbe picos de carga mediante cola y deja mediciones suficientes para evaluar estabilidad y capacidad.

### 3.6 Herramientas y tecnologias

Categoria

Herramientas

Funcion dentro del sistema

Ingesta

Python, Telethon, RabbitMQ

Conexion a Telegram y publicacion desacoplada de mensajes

IA

Hugging Face Transformers, PyTorch

Carga del modelo e inferencia binaria

Persistencia

MongoDB

Almacenamiento operativo de trazas y metadatos

Explotacion

FastAPI, Pydantic

Publicacion de resultados, estadisticas y contratos de consulta

Observabilidad

Prometheus, Grafana, psutil

Metricas temporales, paneles y snapshots de recursos

Visualizacion

React

Consulta funcional de runs, mensajes y benchmarks

Integracion

Docker, docker compose, pytest, GitHub Actions

Despliegue local y validacion automatizada

### 3.7 Infraestructura y despliegue

El proyecto esta preparado para ejecutarse localmente mediante docker compose con perfiles diferenciados. El stack base incluye mongo, rabbitmq, worker, api y frontend; el perfil observability incorpora Prometheus y Grafana; y el perfil advanced activa el productor real sobre Telegram. Esta organizacion facilita demostracion, depuracion y defensa del sistema sin obligar a levantar siempre todos los componentes.

El endurecimiento minimo introducido en docker-compose.yml evita la exposicion publica de MongoDB, deja la API protegida por X-API-Key, restringe CORS a origenes concretos y obliga a mantener

Grafana con autenticacion. La consola de RabbitMQ se conserva porque resulta util en la demo, pero se entiende como un recurso de laboratorio local, no como interfaz abierta de produccion.

La topologia definida en docker-compose.yml despliega siete servicios principales cuando se activa la observabilidad completa. mongo conserva datos en un volumen dedicado; rabbitmq desacopla productor y consumidor; worker ejecuta la inferencia y expone metricas propias; api sirve resultados y agregados; frontend ofrece consulta funcional; prometheus centraliza scraping; y grafana presenta paneles de rendimiento y capacidad. El productor de Telegram permanece separado como servicio opcional.

Esta separacion aporta ventajas practicas desde la perspectiva de sistemas. RabbitMQ se ha escogido frente a Kafka porque el patron productor-consumidor del TFG es sencillo, la consola facilita explicar el desacoplamiento y la sobrecarga operativa es menor para un prototipo academico. MongoDB, por su parte, encaja bien como almacenamiento operativo porque permite persistir documentos heterogeneos con trazas y metadatos sin forzar un esquema rigido.

Tambien conviene destacar que la observabilidad ya no se resuelve con un JSON servido a Grafana. Prometheus actua como fuente real de metricas temporales, scrapea tanto la API como el worker y permite construir paneles de throughput, latencia p95, profundidad de cola, CPU y memoria. El benchmark controlado completa esa observabilidad con escenarios repetibles de 1, 5, 10 y 20 mensajes por segundo.

## 4. Desarrollo e integracion del sistema

### 4.1 Ingesta y gestion de sesion con Telegram

El punto de entrada operativo se reparte ahora entre `producer.py`, `worker.py` y `main.py`. El productor valida `TELEGRAM_API_ID`, `TELEGRAM_API_HASH` y `TELEGRAM_PHONE`, crea la sesion de Telethon y escucha eventos `NewMessage` para transformar cada mensaje en un envelope ligero apto para publicarse en RabbitMQ.

Una vez iniciada la sesion, la ingesta queda desacoplada del

procesamiento. Esta decision permite absorber picos temporales y separar claramente la responsabilidad de captura de la responsabilidad de inferencia. `main.py` conserva un modo legacy para no romper compatibilidad con el MVP anterior, pero la arquitectura defendida en esta memoria es la ruta `producer -> rabbitmq -> worker`.

## 4.2 Preprocesado e inferencia

El preprocesado actual se centraliza en `pipeline.py` e incluye normalizacion basica del texto, reduccion de espacios, conversion a minusculas, limpieza de caracteres y deteccion del idioma cuando la informacion esta disponible. Sobre esa base se ejecuta la inferencia con el modelo binario integrado y se generan tiempos de preprocesado e inferencia por cada mensaje.

La salida principal del modelo sigue siendo `score_1`, interpretado como probabilidad de amenaza. La decision binaria final se calcula aplicando `THRESHOLD`. Para mantener compatibilidad con el estado previo del proyecto se conserva `latency_ms`, pero ahora se acompaña de `preprocess_latency_ms`, `inference_latency_ms`, `db_write_latency_ms`, `queue_wait_latency_ms` y `end_to_end_latency_ms`.

Aunque esta memoria no desarrolla el entrenamiento con el detalle propio del TFG de Informatica, si conviene explicar de donde sale el modelo integrado. `model_loader.py` intenta reutilizar artefactos locales entrenados y, cuando existen, recupera la misma configuracion documentada en `training_metadata.json` para mantener coherencia entre evaluacion offline e inferencia operacional.

La evidencia disponible en `docs/training_metadata.json`, `scripts_experimentos/`, `datos_entrenamiento/` y `modelos_entrenados/` muestra además que el modelo no se ha seleccionado de forma casual. El componente de IA queda integrado como una pieza concreta del sistema, no como un servicio externo opaco.

## 4.3 Persistencia y minimizacion de datos

La coleccion MongoDB no guarda por defecto el texto original. El

documento base incluye:

run\_id y source;

created\_at\_utc, source\_received\_at\_utc, queued\_at\_utc y  
persisted\_at\_utc;

user\_hash, chat\_hash y message\_id;

msg\_sha256, channel e idioma;

pred, score\_1 y threshold;

preprocess\_latency\_ms, inference\_latency\_ms, db\_write\_latency\_ms y  
end\_to\_end\_latency\_ms;

queue\_wait\_latency\_ms y queue\_depth;

cpu\_percent, rss\_bytes, vms\_bytes y gpu\_memory\_bytes cuando  
aplica;

ok, error, hf\_model, model\_source y device.

Esta estrategia ofrece un equilibrio util entre trazabilidad y privacidad. El sistema conserva una huella del contenido, identifica la ejecucion concreta y permite auditar la inferencia sin depender del texto original salvo configuracion explicita. Ademas, las nuevas latencias por etapa y snapshots de recursos convierten cada documento persistido en una evidencia tecnica mas rica.

En la practica, la persistencia tambien incorpora decisiones de mantenimiento que suelen quedar fuera de una memoria centrada solo en modelos. ensure\_indexes() crea indices por run\_id, msg\_sha256 y timestamps, y puede aplicar retencion automatica sobre created\_at\_utc para limitar el historico operativo si asi se configura.

La pseudonimizacion se realiza mediante privacy\_utils.py, que genera user\_hash y chat\_hash con sha256 y una sal configurable. Junto a ello, las variables STORE\_MSG\_ORIGINAL, STORE\_CHANNEL\_NAME y STORE\_USER\_FIELDS permiten graduar el nivel de detalle persistido y mantener el principio de



minimizacion de datos.

#### 4.4 Versionado de artefactos por ejecucion

Una mejora estructural importante fue dejar de tratar la evaluacion como un estado global unico. A partir de la revision, `evaluate.py` crea artefactos en `reports/runs/<run_id>/` y el benchmark controlado hace lo mismo en `reports/benchmarks/<benchmark_id>/`. De esta forma, cada evidencia queda asociada a una ejecucion concreta y deja de sobrescribirse al repetir experimentos.

Esta decision afecta a toda la capa de explotacion:

la API puede servir la matriz de confusion o el analisis por umbral correctos para un `run_id` concreto;

el dashboard React consulta datos coherentes con la ejecucion seleccionada y lista benchmarks historicos;

los scripts de validacion y benchmark pueden comparar artefactos sin perder la evidencia previa.

La organizacion de artefactos en `reports/runs/<run_id>/` y `reports/benchmarks/<benchmark_id>/` tiene ademas una ventaja arquitectonica clara: separa el historico canonico de la copia rapida del ultimo estado. `reporting.py` mantiene los directorios versionados y, al mismo tiempo, deja espejos resumidos en la raiz de `reports/` para consumo operativo.

En terminos de integracion, `run_id` y `benchmark_id` actuan como identificadores comunes entre el clasificador offline, el benchmark, la API, el dashboard web y la memoria del proyecto. Gracias a ello, una consulta o una captura de pantalla puede trazarse facilmente hasta el artefacto que la respalda.

#### 4.5 API y capa de consulta

La API `FastAPI` actua como interfaz intermedia entre la persistencia, los artefactos versionados y las herramientas de explotacion. Expone endpoints para salud, ejecuciones, resumen de metricas, umbrales, matriz de confusion, mensajes persistidos, estadisticas

agregadas, benchmarks y metadatos de entrenamiento. Además publica `/metrics` como superficie de scraping para Prometheus.

La aplicación utiliza una construcción perezosa (`LazyFastAPIApp`) para evitar que la importación falle cuando aún no se han definido ciertas variables de entorno. Esa decisión mejora la robustez del despliegue y facilita tanto el arranque en contenedores como la ejecución de pruebas. En paralelo, `worker.py` expone sus propias métricas de proceso y de pipeline para completar la observabilidad.

Más que una lista de endpoints, la API debe entenderse como el contrato operativo del sistema. `api/app.py` centraliza autenticación simple mediante dependencia, validación de parámetros y serialización de respuestas; `api/services.py` agrega latencias, percentiles, throughput y consumo medio de recursos a partir de las trazas persistentes.

La propia configuración refuerza el enfoque de despliegue defendible. `api/settings.py` exige `API_KEY`, define orígenes CORS concretos y resuelve rutas hacia `reports/` y `docs/training_metadata.json`. Con ello se separa claramente la consulta funcional de la observabilidad temporal que asume Prometheus.

#### 4.6 Dashboard React y Grafana

La capa web permite consultar rápidamente el estado de las ejecuciones, los mensajes persistentes y los benchmarks disponibles desde una interfaz accesible. React se utiliza como frontend operativo de consulta, mientras que Grafana se reserva para el seguimiento temporal de la carga y la salud del pipeline sobre datos de Prometheus.

Figura 3. Vista resumida del dashboard React para exploración de runs, mensajes y benchmarks.

Además del resumen ejecutivo y la vista de mensajes, el dashboard incorpora una pantalla de rendimiento que facilita revisar la evolución de las métricas principales y contrastarla con los benchmarks guardados. Esto resulta útil durante la defensa porque permite relacionar una ejecución offline concreta con el

comportamiento del sistema bajo carga controlada.

Figura 5. Vista del dashboard React orientada al seguimiento de latencias por etapa y benchmarks.

Figura 6. Panel de Grafana sobre Prometheus para monitorización operativa del pipeline.

La coexistencia de ambas capas refuerza la idea de pipeline explotable: la API sirve como contrato técnico común para consulta funcional y React se orienta a revisión por ejecución; Prometheus y Grafana, en cambio, se especializan en series temporales, percentiles, profundidad de cola y consumo de recursos.

El dashboard React y Grafana no duplican exactamente el mismo papel. El primero permite inspeccionar ejecuciones concretas, revisar mensajes, filtrar por predicción o score y consultar benchmarks históricos. Grafana ofrece paneles vivos de throughput, latencia p95, CPU, memoria y cola, sin depender de lecturas manuales sobre MongoDB.

Este detalle de integración es relevante porque evita tanto el acceso directo de Grafana a MongoDB como la dependencia del antiguo enfoque JSON para monitorización. La API queda protegida para consulta funcional y Prometheus se consolida como la fuente especializada de observabilidad.

#### 4.7 Validación integrada

El proyecto incorpora varios niveles de verificación:

pruebas pytest para la API y la capa de servicios;

evaluación offline reproducible por `run_id`;

simulación de casos operativos y benchmark controlado con 1, 5, 10 y 20 mensajes por segundo;

runner de integración y comprobaciones básicas en GitHub Actions.

La observacion practica de tutoria sobre pytest -q frente a python -m pytest -q ha quedado resuelta y, ademas, la validacion ya no se apoya solo en tests unitarios. scripts/run\_phase5\_checks.py comprueba artefactos clave, benchmark\_pipeline.py genera evidencias cuantitativas y el workflow de GitHub Actions ejecuta una bateria minima reproducible sobre ambos TFGs.

## 5. Evaluacion y analisis de resultados

### 5.1 Metricas principales

La evaluacion offline de referencia, almacenada en reports/runs/63c1d8a6-613a-4971-b3bf-dc0fe2907001/metrics.json, ofrece los siguientes resultados sobre 100 muestras balanceadas:

accuracy = 0.63

tasa\_error\_prueba = 0.37

precision\_pos = 0.5823

recall\_pos = 0.92

f1\_pos = 0.7132

roc\_auc = 0.8684

average\_precision = 0.8951

La tasa de error en datos de prueba se obtiene directamente como 1 - accuracy, por lo que el sistema falla en 37 de cada 100 muestras del conjunto offline utilizado en la evaluacion. Este dato obliga a interpretar el clasificador como una primera criba y no como una decision final automatica.

Estas metricas no deben leerse de forma aislada. En este TFG interesa, ademas, comprobar que el pipeline mantiene trazabilidad, estabilidad y tiempos de respuesta aceptables cuando se integra como sistema. Por eso la evaluacion offline se complementa con benchmark controlado y monitorizacion de recursos.

Figura 7. Analisis por umbral utilizado para comparar precision, recall, F1 y accuracy.

## 5.2 Interpretacion del umbral operativo

El analisis por umbral muestra que el mejor F1 de la evaluacion aparece en torno a 0.45, donde precision\_pos, recall\_pos, f1\_pos y accuracy se estabilizan en torno a 0.82. Sin embargo, el sistema opera con THRESHOLD = 0.05 para priorizar recall y reducir la probabilidad de dejar pasar mensajes potencialmente peligrosos.

En otras palabras:

con 0.05 se detectan 46 de 50 amenazas reales;

a cambio, aparecen 33 falsos positivos en la clase negativa;

esa decision es aceptable si el sistema se usa como primera criba para revisiones posteriores y se prioriza sensibilidad frente a precision.

## 5.3 Matriz de confusion y distribucion

La matriz de confusion del ultimo run\_id fue:

verdaderos negativos: 17

falsos positivos: 33

falsos negativos: 4

verdaderos positivos: 46

Figura 8. Matriz de confusion correspondiente a la evaluacion offline del sistema.

Figura 9. Metricas globales obtenidas en la evaluacion offline del modelo.

Figura 10. Distribucion de clases utilizada en la evaluacion offline.

El comportamiento es consistente con un detector sensible: el sistema recupera muchas amenazas, pero genera un volumen apreciable de revisiones adicionales. Desde el punto de vista del pipeline, esto refuerza la necesidad de conservar trazabilidad y filtros posteriores para que la carga humana sea asumible.

#### 5.4 Evidencia de validacion integrada

La validacion cuantitativa ya no se limita a confirmar que existen artefactos. El benchmark controlado mas reciente, almacenado en `reports/benchmarks/bench-20260402T153415Z-b54ed0d2/benchmark_summary.json`, ejecuta escenarios de 5 segundos por carga objetivo y permite contrastar throughput, latencia y recursos del proceso sobre la misma base tecnica del worker.

Escenario 1 msg/s: throughput real 1.24 msg/s, latencia media 171.71 ms, p95 513.19 ms y tasa de error 0%.

Escenario 5 msg/s: throughput real 5.16 msg/s, latencia media 50.04 ms, p95 57.95 ms y tasa de error 0%.

Escenario 10 msg/s: throughput real 10.09 msg/s, latencia media 52.29 ms, p95 63.99 ms y tasa de error 0%.

Escenario 20 msg/s: throughput real 19.08 msg/s, latencia media 52.4 ms, p95 66.06 ms y tasa de error 0%.

El escenario mas exigente, ejecutado sin escritura en MongoDB para aislar el coste de inferencia y orquestacion, sostuvo 19.08 msg/s con latencia media 52.4 ms, p95 66.06 ms, uso medio de CPU del 773.57% del proceso y RSS medio de 0.88 GiB. Junto a ello, `scripts/run_phase5_checks.py` y la CI del repositorio verifican la presencia de artefactos y la salud basica del sistema.

#### 5.5 Limitaciones detectadas

Durante la evaluacion se han identificado varias limitaciones:

El umbral operativo bajo mejora recall, pero genera un numero alto de falsos positivos.

El benchmark se ha ejecutado en un unico nodo local y, en su forma

de referencia, sin escritura real en MongoDB, por lo que no sustituye a una prueba distribuida completa.

RabbitMQ desacopla productor y worker, pero el proyecto todavia no incorpora politicas automaticas de reintento, DLQ ni escalado horizontal gestionado.

Prometheus aporta observabilidad real, aunque los dashboards y alertas siguen teniendo alcance academico y no de operacion 24x7.

El sistema actual clasifica de forma binaria y mantiene un preprocesado deliberadamente conservador; podria enriquecerse con mas contexto, explicabilidad o categorias de amenaza.

## 6. Conclusiones y lineas futuras

El trabajo desarrollado cumple su objetivo principal: construir un pipeline funcional y defendible para detectar mensajes de phishing o actividad sospechosa en Telegram desde una perspectiva propia de Ingenieria de Computadores. La arquitectura producir -> RabbitMQ -> worker -> MongoDB -> API -> React, reforzada con Prometheus y Grafana, permite explicar con claridad como se integra cada pieza y que evidencia tecnica deja a su paso.

El proyecto demuestra que es posible combinar:

captura automatizada de mensajes y publicacion en cola;

inferencia con un modelo de lenguaje y latencias por etapa;

almacenamiento trazable y minimizado;

API reutilizable y metricas Prometheus;

visualizacion operativa y benchmark controlado;

validacion integrada por artefactos y pruebas.

Ademas, la revision de tutoria ha servido para mejorar aspectos esenciales de calidad tecnica: desacoplamiento entre ingesta e inferencia, fuente real de observabilidad, medicion de CPU y memoria, benchmark reproducible y una topologia de despliegue

mas defendible. Esa evolucion es precisamente una de las aportaciones mas fuertes del TFG.

La memoria permite justificar ademas que la IA integrada responde a un proceso previo de entrenamiento documentado y no a la seleccion casual de un modelo externo. Sin embargo, el foco academico del trabajo no se desplaza hacia la busqueda del mejor clasificador, sino hacia la integracion de ese clasificador dentro de un sistema medible y operable.

Desde el punto de vista cuantitativo, la tasa de error de prueba del 37% confirma que el sistema no debe interpretarse como un clasificador final autonomo, sino como una primera criba. Al mismo tiempo, el benchmark muestra que el pipeline puede sostener en local aproximadamente 19.08 mensajes por segundo con latencias medias cercanas a 52.4 ms, lo que refuerza su viabilidad como prototipo de sistemas.

Como lineas de trabajo futuro, resultaria razonable:

explorar umbrales adaptativos o politicas por contexto;

ampliar el worker con consumidores paralelos, reintentos y colas de error;

enriquecer la observabilidad con alertas, SLOs y benchmark con escritura real en MongoDB;

incorporar mecanismos mas ricos de explicabilidad y clasificacion multiclase.

En su estado actual, el TFG queda mejor diferenciado respecto al TFG de fake news: aqui el peso esta en la arquitectura del pipeline, la integracion de componentes, la observabilidad y la evaluacion del sistema como infraestructura operativa, no en la construccion del dataset ni en la experimentacion principal sobre modelos.

## Bibliografia

Akinyele, A. R., Ajayi, O. O., Munyaneza, G., Ibecheozor, U. H., & Gopakumar, N. (2024). Leveraging Generative Artificial Intelligence for cybersecurity: Analyzing diffusion models in detecting and



mitigating cyber threats. GSC Advanced Research and Reviews, 21(2), 1-14.

Chatzimarkaki, G., Karagiorgou, S., Konidi, M., Alexandrou, D., Bouras, T., & Evangelatos, S. (2023). Harvesting large textual and multimedia data to detect illegal activities on dark web marketplaces. IEEE International Conference on Big Data, 4046-4051.

Duggineni, S. (2024). AI and machine learning applications in industrial automation and cybersecurity. Journal of Artificial Intelligence & Cloud Computing, 2(4), 1-5.

Elbes, M., Hendawi, S., AlZu'bi, S., Kanan, T., & Mughaid, A. (2023). Unleashing the full potential of artificial intelligence and machine learning in cybersecurity vulnerability management. International Conference on Information Technology, 276-283.

Europol. (2022). Internet Organised Crime Threat Assessment (IOCTA) 2022. European Union Agency for Law Enforcement Cooperation.

Gartner. (2023). Top Strategic Technology Trends for 2023. Gartner, Inc.

Guo, Y., Wang, D., Wang, L., Fang, Y., Wang, C., Yang, M., Liu, T., & Wang, H. (2024). Beyond App Markets: Demystifying underground mobile app distribution via Telegram. Proceedings of the ACM on Measurement and Analysis of Computing Systems, 8(3), 1-25.

Hummelholm, A., Iturbe, E., Krichen, M., et al. (2023). Artificial intelligence and cyber threat monitoring: Emerging approaches for automated detection. Journal of Cybersecurity Engineering, 5(2), 45-63.

INCIBE. (2024). Buenas practicas de ciberseguridad para la deteccion y gestion de phishing. Instituto Nacional de Ciberseguridad.

Roy, S., et al. (2024). Characterizing cybercriminal ecosystems on

Telegram at scale. Security and Privacy Workshops, 1-12.